



PRODUCT REQUIREMENTS DOCUMENT

Kali Tools GUI

ממשק גרפי ושכבת סוכני AI מסוג Purple-Team לכלי Kali Linux

אבטחת מידע

גרסה 1.0

מסמך אפיון מוצר

מגיש: **הראלי דודאי**

ת.ז.: 027225721

דוא"ל: hareli26@gmail.com

מוסד: **מכללת HackerU**

קורס: אדריכלות AI

מנחה: שחף

תאריך: יולי 2026

תוכן עניינים

1. תקציר מנהלים
2. רקע ומוטיבציה
3. מטרות המוצר
4. קהל יעד ופרסונות
5. דרישות פונקציונליות
6. דרישות לא-פונקציונליות
7. סקירת ארכיטקטורה
8. תכונות המוצר בפירוט
9. מדדי הצלחה (KPIs)
10. גבולות המערכת (Scope)
11. סיכונים והנחות
12. מפת דרכים עתידית

1. תקציר מנהלים

Kali Tools GUI היא מערכת גרפית (Web) המאחדת את עשרות כלי ה-Kali Linux, ורצה על **שרת Kali ייעודי (VPS)** — עם תמיכה גם ב-WSL2 לפיתוח מקומי, ומוסיפה מעליהם שכבת סוכנים חכמה בגישת **Purple Team**: צוות אדום שמאתר פרצות, צוות כחול שמפיק הגנות, סוכן מתווך ביניהם, וסוכן מתזמר שמנצח את התהליך ומדווח — עם למידה מצטברת בין הרצות. ומעל הכול: **שכבת הטעיה (מלכודות דבש)** שמזינה את המערכת ב**תקיפות אמיתיות** מהאינטרנט, וסוגרת **לולאת למידה** שבה מה שתוקפים מנסים בפועל משנה את מה שהסוכנים בודקים.

המערכת פותרת פער מרכזי: כלי הפנטסט חזקים אך דורשים ידע בשורת פקודה ובעשרות דגלים, והתוצאות מפוזרות וקשות לפרשנות. המערכת מגישה אותם דרך טפסים גרפיים, ומעל זה מאפשרת למשתמש לתאר **כוונה** בשפה חופשית ולקבל בדיקה שלמה שמתוכננת, מורצת, מאומתת ומדווחת אוטומטית — כולל המלצות הגנה קונקרטיות.

הפער השני, והעמוק יותר, הוא **שהמערכת למדה רק מתקיפות שהיא עצמה יזמה** — מעגל סגור ומלאכותי. שכבת הטעיה פותחת אותו: מלכודות על שרת מוקרב נפרד קולטות תקיפות אמיתיות, המערכת מסווגת אותן, **מצליבה אותן מול החולשות שהסריקות שלנו מצאו אצלנו**, ומזינה את התובנה חזרה לתכנון. התוצאה היא מעבר מ-"מה עלול להיות רע" ל-"במה תוקפים באמת מנסים לפגוע בנו עכשיו" — תעדוף מבוסס איום אמיתי במקום צ'קליסט תיאורטי.

2. רקע ומוטיבציה

Kali Linux היא הפצת לינוקס המובילה לבדיקות חדירות, המכילה מאות כלים. עם זאת:

הבעיה	ההשפעה	הפתרון במערכת
תחביר מורכב ודגלים רבים לכל כלי	עקומת למידה תלולה, שגיאות	טופס דינמי + תצוגת פקודה חיה
לא ברור באיזה כלי להשתמש לכל מטרה	בזבוז זמן, כיסוי חלקי	סוכן Planner שממפה כוונה→כלים
תוצאות מפוזרות וקשות לפרשנות	קושי להסיק מסקנות	סוכני Reporter ו-Verifier
פער בין מציאת פרצה ליכולת להגן	ממצא ללא פעולה	שכבת Blue Team עם המלצות הגנה

המוטיבציה החינוכית: מערכת המדגימה עקרונות של Red/Blue/Purple Team, ארכיטקטורת מיקרו-שירות פשוטה, בניית פקודות בטוחה, ותכנון מבוסס-ידע — כל זאת ללא תלות בספריות חיצוניות.

3. מטרת המוצר

1. **הנגשה:** להפוך כלי פנטסט מורכבים לנגישים דרך ממשק גרפי אינטואיטיבי בעברית.
2. **אוטומציה חכמה:** לתרגם כוונה בשפה טבעית לתוכנית בדיקה מלאה שמתבצעת מקצה לקצה.
3. **ערך הגנתי:** לא רק לגלות חולשות אלא להפיק לכל ממצא המלצת הגנה, ניטור ומיפוי MITRE ATT&CK.
4. **למידה:** לצבור ידע על ממצאים חוזרים ולשקף מגמות לאורך זמן.
5. **עמידות:** לפעול כשירות ייצור יציב, מאובטח וזמין תמידית.

4. קהל יעד ופרסונות

פרסונה	צורך עיקרי	כיצד המערכת עונה
סטודנט לאבטחת מידע	ללמוד כלים ותהליכים	ממשק מודרך + הסברים לכל כלי ואיום
בודק חדירות מתחיל	להריץ בלי לזכור תחביר	טפסים + Playbooks מוכנים
אנליסט הגנה (Blue)	לדעת כיצד להתגונן	המלצות הגנה, ניטור ו-MITRE לכל ממצא
חובב CTF	מהירות וכיסוי	שרשראות כלים אוטומטיות + דוח מרוכז

5. דרישות פונקציונליות

מזהה	דרישה	עדיפות
FR-01	הצגת קטלוג כלים עם חיפוש, סינון לפי קטגוריה וזיהוי מותקן/לא-מותקן	חובה
FR-02	טופס דינמי לכל כלי עם תצוגת פקודה מתעדכנת בזמן אמת	חובה
FR-03	הרצת כלי והצגת פלט חי במסך תוצאות נפרד, עם עצירה/העתקה/הורדה	חובה
FR-04	התקנת כלי חסר מתוך הממשק	רצוי
FR-05	תרגום כוונה בשפה חופשית לתוכנית שלבים (Planner)	חובה
FR-06	הרצת שלבים ברצף עם אימות אוטומטי של כל שלב (Executor+Verifier)	חובה
FR-07	הפקת דוח Markdown מלא (Reporter) ושמידתו לצפייה חוזרת	חובה
FR-08	מיפוי כל ממצא אדום להמלצת הגנה, ניטור, תצורה ו-MITRE (Blue Team)	חובה
FR-09	צבירת ידע על ממצאים חוזרים בין הרצות (Learning)	חובה
FR-10	דשבורד "מרכז בקרה": מצב סוכנים חי, מודיעין נצבר ויומן פעילות	חובה
FR-11	תצוגת "מוח סוכן" גרפית עם התקרבות סמנטית (semantic zoom)	רצוי
FR-12	הרצה חוזרת של פעילות קיימת עם אישור מפורש	רצוי
FR-13	סוכן מתקן (Remediation): יישום הגנות בפועל על המערכת — עם אישור חובה, גיבוי ולוג	חובה
FR-14	שרת MCP: שליטה מלאה במערכת דרך שיחה עם Claude / כל לקוח MCP	רצוי
FR-15	פריסה בענן מאובטחת עם התחברות Google, אישור משתמשים ולוג ביקורת	חובה
FR-16	מסך ניהול משתמשים (אדמין בלבד): הוספה/הסרה של חשבונות Google מורשים מתוך הממשק, עם טעינה חיה ורישום ביומן	חובה
FR-17	תצוגת גרף Obsidian מובנית: מפת קשרים אינטראקטיבית בין דוחות לאיומים (גרירה, זום, פתיחת פתק) + ייצוא כספת Markdown	רצוי
FR-18	סרגל "ריצה" גלובלי: חיווי חי של המשימה הפעילה בכל מסך (כלי נוכחי, התקדמות) עם חזרה בלחיצה; הריצה נמשכת בשרת ללא תלות בניווט	חובה
FR-19	חוקי זיהוי מוכנים לפריסה (Sigma ל-Suricata + SIEM ל-IDS) לכל איום, בדוח ובחלון פרטי האיום	רצוי
FR-20	מסך היסטוריה + השוואת שתי סריקות של אותה מטרה (נפתר / חדש / עדיין פתוח = מגמת אבטחה)	רצוי
FR-21	עורך Playbooks מהממשק (אדמין): הוספה/עריכה של חוקי סוכן מבוססי-דאטה (placeholders בטוחים, ללא הרצת קוד)	רצוי
FR-22		רצוי

	דף חומרי למידה: מה המערכת למדה + Q&A חינוכי לכל סוג איום (מהו, הגנה, זיהוי, תיקון, MITRE)	
רצוי	תצוגת גלקסיה תלת-מימדית לכספת Obsidian: כל כוכב = נתון; סיבוב, טיסה וגרירת כוכבים (קנבס טהור, אפס ספריות)	FR-23
חובה	הרשאות תפקידים (RBAC): admin (הכל) · operator (הרצה) · viewer (צפייה); אכיפה בצד שרת + התאמת הממשק	FR-24
רצוי	אינטגרציית LLM מקומי (Ollama): ניסוח דוחות + תקציר AI, עם בדיקת נגישות ונפילה בחן למנוע מבוסס-חוקים	FR-25

6. דרישות לא-פונקציונליות

מזהה	דרישה
NFR-01	אבטחה: בניית פקודות כמערך <code>argv</code> ללא <code>shell</code> (הגנה מהזרקה); האזנה ל-127.0.0.1 בלבד; טוקן אופציונלי.
NFR-02	אפס תלויות: צד השרת בספריית פייתון סטנדרטית בלבד — עובד על Kali מינימלי.
NFR-03	זמינות: שירות <code>systemd</code> עם הפעלה אוטומטית והתאוששות מקריסה.
NFR-04	עמידות: טיפול גלובלי בשגיאות, גיזום זיכרון, לוגים, נקודת בריאות.
NFR-05	שפה ונגישות: ממשק RTL מלא בעברית, ערכת נושא כהה.
NFR-06	ביצועים: זמן ממחשבה עד הרצה > 30 שניות; פלט מוזרם עם מגבלת גודל.
NFR-07	שקיפות: הידע מבוסס-חוקים ניתן לביקורת ולעריכה (JSON/פייתון).
NFR-08	ביקורתיות (Auditability): כל פעולה (הרצה/תיקון/התקנה) נרשמת עם המשתמש שביצע אותה.
NFR-09	בטיחות תיקון: תיקונים מסוכנים (SSH/firewall) אינם מיושמים אוטומטית; גיבוי לפני כל שינוי.

7. סקירת ארכיטקטורה



איור 1 — סקירת ארכיטקטורה. פירוט מלא במסמך התכן הנלווה.

8. תכונות המוצר בפירוט

8.1 מצב "כלים" (שימוש ידני)

קטלוג של 72 כלים ב-11 קטגוריות (Recon, DNS, Network, Web, SMB, Passwords, SSL, Exploitation, Forensics, Wireless, Utilities). לכל כלי כרטיס הסבר ("מה זה ומה אפשר לעשות") המוצג במסך הטופס, וקישור ישיר (`#tool-<id>`). בחירת כלי פותחת טופס יכולות דינמי; הפקודה נבנית בזמן אמת; ההרצה מוצגת במסך תוצאות נפרד.

8.2 מצב "עוזר AI" (שכבת סוכנים)

המשתמש מתאר כוונה (למשל "בדיקת חדירות לאתר") ומטרה. הסוכנים פועלים ברצף: **Planner** (כוונה→תוכנית) ← אישור המשתמש → **Executor** (הרצה) → **Verifier** (אימות + חילוץ ממצאים) → **Reporter** (דוח).

8.3 שכבת Purple Team

הצוות האדום מאתר פרצות, סוכן ה-**Broker** ממפה כל ממצא לכלל הגנה, הצוות ה-**כחול** מפיק הגנות/ניטור/תצורה, מיפוי MITRE, **וחוקי זיהוי מוכנים לפריסה — Sigma (SIEM) ו-Suricata (IDS)**, וה-**Orchestrator** מתזמר ומפיק דוח Purple. סוכן ה-**Learning** צובר חתימות ממצאים בין הרצות.

8.4 מרכז בקרה (Dashboard)

מבט חי על מצב הסוכנים, מודיעין נצבר (חלוקת חומרה, איומים נפוצים), ויומן פעילות בסגנון מגזין. לחיצה על סוכן פותחת **תצוגת "מוח"** גרפית עתידנית עם התקרבות סמנטית החושפת שכבות יכולות.

8.5 סוכן מתקן (Remediation Agent)

מיישם בפועל את ההגנות שהצוות הכחול ממליץ (למשל התקנת fail2ban, עדכוני אבטחה אוטומטיים) — תמיד לאחר **אישור מפורש**, עם גיבוי אוטומטי של קונפיגים ורישום ביומן הביקורת. תיקונים בעלי סיכון גבוה (כיבוי סיסמת SSH, שינוי firewall) מוצגים כהנחיה בלבד ואינם רצים אוטומטית, כדי למנוע נעילה.

8.6 שכבת הטעיה — מלכודות דבש (Deception)

עד כה הצוות הכחול למד רק מתקיפות שאנחנו יזמנו — מעגל סגור ומלאכותי. שכבת ההטעיה פותחת אותו לעולם האמיתי: **מלכודת ווב באינטראקציה נמוכה** (`web.dudaei.com`) מגישה אתר חברה מזויף זרוע פיתיונות (`.env`) עם מפתחות מזויפים, `.git/config`, `dump` מזויף של DB, טופס התחברות שקולט את האישורים שנוסו, ו-`endpoint` שנראה פגיע ל-SQLi ומחזיר שגיאת MySQL מזויפת כדי שהתוקף יעמיק ויחשוף עוד מהטכניקה שלו).

הפרדה פיזית: המלכודת רצה על **שרת מוקרב ונפרד** — לעולם לא על הפרודקשן. מלכודת היא שרת שמזמינים לפרוץ אליו; אילו חלקה מכונה עם המערכת, פריצה מוצלחת הייתה נוחתת על מסד הנתונים, יומן הביקורת ותצורת ה-OAuth. המלכודת **אינה מריצה דבר** (אין `shell`, אין `eval`, אין מנוע SQL — כל תשובה

היא מחרוזת קבועה), **אינה מסווגת** (ניתוח רק בפרודקשן, כך שפריצה אליה לא מרעילה את המסווג), **ואינה מחזיקה שום מפתח לפרודקשן** – הפרודקשן מושך ממנה, לא להפך.

8.7 שני בסיסי ידע – והצלבתם

אירוע ממלכודת הוא הפוך סמנטי ממצא של הצוות האדום, ולכן הוא נשמר בבסיס ידע נפרד:

בסיס ידע	מקור	מה הוא טוען
DEFENSE_KB	הצוות האדום שלנו	"למטרה הזו יש חולשה" (תנוחה)
ATTACK_KB	מלכודות הדבש	"מישהו ניסה את הטכניקה הזו" (התנהגות)

הזרמת אירועי מלכודת דרך DEFENSE_KB הייתה רושמת טענה שגויה ("יש לי חולשת הזרקה" במקום "מישהו ניסה להזריק") ומזהמת את סטטיסטיקות התנוחה. ATTACK_KB מכיל 19 טכניקות (Log4Shell, Shellshock, SSRF למטא-דאטה של הענן, XSS, Webshell, Path Traversal, RCE, SQLi, ציד סודות, תקיפת אישורים, מיפוי CMS, סריקה אוטומטית, חיפוש נתיבים) – לכל אחת הסבר, הגנות, מיפוי MITRE וחוקי Sigma/Suricata מוכנים.

ההצלבה היא הערך: טכניקה שתוקפים מנסים בפועל, נגד חולשה שהסריקה שלנו מצאה אצלנו – אינה "עוד ממצא ברשימה" אלא **תעדוף מבוסס איום אמיתי**. חומרה לבדה מדרגת לפי כמה עלול להיות רע; ההצלבה מוסיפה האם מישהו באמת מנסה.

8.8 לולאת הלמידה – הסוכנים לומדים מהתקיפות

המודיעין אינו רק תצוגה: הוא **מוזן חזרה לסוכנים ומשנה את התנהגותם**.

- תכנון (Planner):** טכניקות שנצפו מוסיפות שלבי בדיקה שה-playbook הקבוע לא חשב עליהם. בפועל: תוכנית של 5 שלבים גדלה ל-8 והוסיפה sqlmap ("תוקפים מנסים 47 x SQLi"), gobuster ("מחפשים 18 x env") ו-wpscan ("ממפים 9 x CMS"). הצוות האדום מפסיק לבדוק צ'קליסט קבוע ומתחיל לבדוק את מה שהאינטרנט זורק עלינו השבוע.
- תכנון AI:** המודיעין החי מוזרק לפרומפט של ה-LLM, כך שהוא בוחר כלים לפי איומים אמיתיים.
- דוח הצוות הכחול:** חולשה תחת תקיפה פעילה מסומנת ומתועדפת (🔥) המלכודות קלטו 47 ניסיונות – זו אינה חולשה תיאורטית").

גבולות הלולאה (חשוב): נתוני מלכודת הם קלט של תוקף. לכן: השפעה רק על מה בודקים, לעולם לא על מה שעושים לעצמנו; רק כלי גילוי/מיפוי (כלים תוקפניים כמו hydra מוחרגים במכוון – תוקף שיזהה את הלולאה יוכל להציף brute-force וגרום לנו לנעול את חשבונותינו); נדרשות 3 תצפיות לפחות כדי שבקשה מזויפת בודדת לא תטה את התכנון; עד 3 שלבים נוספים; כל שלב מסומן במודיעין שגרם לו; והמשתמש מאשר את התוכנית לפני הרצה. ואף פעם אין תיקון אוטומטי מנתוני מלכודת.

8.9 שרת MCP

חושף את המערכת ככלים ש-Claude (או כל לקוח MCP) יכול להפעיל בשיחה טבעית: להריץ כלים, לתכנן ולהריץ בדיקות, לקבל דוחות ומודיעין, ולהחיל תיקונים (עם אישור). מאפשר שליטה ואוטומציה מתוך צ'אט.

8.10 פריסה בענן מאובטחת

פריסה על VPS מאחורי **Caddy** (HTTPS אוטומטי) → **oauth2-proxy** (התחברות **Google** + רשימת מיילים מאושרים) → האפליקציה (127.0.0.1 בלבד). האדמין **מאשר משתמשים** בקובץ, וכל גישה ופעולה נרשמות בלוג ביקורת תלת-שכבתי.

8.8 ניהול משתמשים (מסך אדמין)

מסך ייעודי הגלוי **רק לאדמין** (מזוהה לפי כתובת ה-Google שלו), לניהול רשימת ההרשאות של **oauth2-proxy** מתוך הממשק: **הוספה/הסרה** של חשבונות Google מורשים בלחיצה. השינויים נטענים **חיה** (ללא restart), עם אימות תקינות אימייל, מניעת נעילה עצמית (אי אפשר להסיר את האדמין) ורישום כל שינוי ביומן הביקורת. כל הפעולות מוגנות בצד השרת (403 למי שאינו אדמין).

8.9 תצוגת גרף Obsidian מובנית

תצוגת **Graph View** אינטראקטיבית בתוך המערכת: הדוחות והאיומים מוצגים כצמתים מחוברים (force-directed) — צומת מרכזי (MOC), דוחות ואיומים צבועים לפי חומרה. תמיכה ב**גרירה**, **זום/הזזה**, **חיפוש צמתים ולחיצה לפתיחת הפתק המלא**. במקביל נשמרת **כספת Obsidian** אמיתית (Markdown + MOC + wikilinks + frontmatter) המתעדכנת אוטומטית אחרי כל בדיקה וניתנת לפתיחה ב-Obsidian הדסקטופ.

8.10 סרגל ריצה גלובלי (מעקב חי)

מכיוון שהמשימות רצות **בשרת** (threads) ואינן תלויות בניווט, נוסף **סרגל "ריצה" גלובלי** המופיע בכל המסכים כל עוד משימה פעילה. הוא מציג בזמן אמת **מה הסוכן עושה כרגע** (הכלי הנוכחי, שלב X מתוך Y, פאזה אדום/כחול ב-Purple) ומאפשר **חזרה לתצוגה המלאה בלחיצה**. הסרגל מתחבר מחדש גם לאחר רענון דף, כך שהמשתמש לעולם לא "מאבד" משימה פעילה.

9. מדדי הצלחה (KPIs)

מדד	יעד
זמן מכוונה עד הרצה	פחות מ-30 שניות
אחוז כלים הניתנים להרצה ללא CLI ידני	100% מהקטלוג
דוח מלא בסיום כל משימה	אוטומטי, 100%
המלצת הגנה לכל ממצא מזוהה	מיפוי מלא ל-DEFENSE_KB
זמינות השירות	הפעלה אוטומטית + התאוששות מקריסה

10. גבולות המערכת (Scope)

בתוך הגבולות: מערכת רב-משתמשית עם הרשאות תפקידים; הרצת כלי Kali; שכבת סוכנים מבוססת-חוקים (עם ניסוח LLM אופציונלי); דשבורד, היסטוריה, למידה וכספת ידע.

מחוץ לגבולות (גרסה נוכחית): ריבוי ארגונים/דיירים (multi-tenant); כלים אלחוטיים חיים (שרת/VM ללא גישת monitor mode לחומרה אלחוטית).

הערה: אימות מרובה-משתמשים (Google + allowlist), **הרשאות תפקידים (RBAC)** ומסד נתונים מנוהל (SQLite) — מומשו וכבר בתוך הגבולות.

11. סיכונים והנחות

סיכון / הנחה	מיטיגציה
שימוש לרעה כנגד מטרות ללא רשות	אזהרות בממשק ובדוחות; שלב אישור לפני כל הרצה
כלי איטי תוקע משימה	מגבלת זמן לכל שלב (timeout) והמשך אוטומטי
הרצה כ-root	האזנה ל-localhost בלבד; פקודות argv ללא shell
כלים אלחוטיים בשרת/VM	נתמך רק (aircrack-ng (offline); שאר האלחוטי מחוץ לגבולות
הרצה עם פרטי משתמש שגויים (RBAC)	אכיפה בצד שרת (403) בכל נקודות הקצה; הממשק בלבד אינו סומך

12. מפת דרכים עתידית

הושג מאז הגרסה הראשונית: סוכן מתקן (Remediation), שרת MCP, פריסה בענן מאובטחת עם התחברות Google ולוג ביקורת (כולל תיעוד כניסות), מסד נתונים SQLite, ניהול משתמשים מהממשק, תצוגת גרף Obsidian מובנית, סרגל ריצה גלובלי, **חוקי זיהוי Sigma/Suricata** לכל איום, **ומסך היסטוריה עם השוואת סריקות** (מגמת אבטחה לאורך זמן).

👉 **שכבת הטעיה (Deception) — מומשה: שתי מלכודות** חיות על שרת מוקרב נפרד (web.dudaei.com) — מלכודת ווב ומלכודת MySQL **ייעודית (פורט 3306)** שמדברת את פרוטוקול MySQL, מקבלת כל התחברות וקולטת את הפלייבוק שאחרי הכניסה (RCE, INTO OUTFILE, UDF, שאיבת משתמשים). בסיס ידע שני של **19 טכניקות תקיפה (ATTACK_KB)**, הצלבה מול תנוחת האבטחה שלנו, **ולולאת למידה סגורה** שבה המודיעין משנה בפועל את תכנון הסוכנים ואת תעדוף הדוח.

✅ **כל סעיפי מפת הדרכים המקורית מומשו.** כיווני המשך אפשריים:

- מלכודות נוספות: SSH, Redis, SMB — כל אחת מרחיבה את **ATTACK_KB** באותו צינור.
- שיתוף מודיעין בין התקנות (threat sharing) — מלכודת של אחד מחסנת את כולם.
- תמיכה במספר ארגונים/דיירים (multi-tenant) ו-SSO ארגוני.

הערות: **עורך Playbooks (data-driven)**, **דף חומרי למידה, LLM מקומי (Ollama)** — כולל **תכנון AI** שבו ה-LLM בוחר בעצמו את הכלים, **הרשאות תפקידים (RBAC): admin / operator / viewer**, **ושכבת מלכודות דבש — מומשו.**

— סוף מסמך ה-PRD · הראלי דודאי · מכללת 2026 · HackerU —